DOCUMENTATION · PERFORMANCE

Performance, v4: what 0.43 changed on PostgreSQL

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=PERFORMANCE_V4. This copy was made on 4 November 2026.

The case is v3's: pgbench's tables at scale 100, saved queries by primary key, a 100-row page and a single-row write, with a plain key and a governed one (a table allow-list and a masking rule). v4 measures 0.43 against 0.42 with v3's harness and method.

The short answer. One gunicorn worker, as in the Docker image, 32 concurrent callers:

	V0.42.0	0.43	0.43 + METADATA CACHE
Lookup, plain key	786/s	852 (+8%)	1,012 (+29%)
Lookup, governed key	738	809 (+10%)	964 (+31%)
100-row page	682	725 (+6%)	835 (+22%)
Write	793	850 (+7%)	1,002 (+26%)
One caller, lookup: p50	1.36 ms	1.27 ms	1.00 ms
One caller, lookup, **PostgreSQL 0.5 ms away**: p50	6.58 ms	**4.39 ms**	**1.60 ms**

- **Across a network, a call is 2.2 ms faster on 0.43, and 5 ms faster with the cache:** 4 times as fast as 0.42. The data query is one round trip instead of four (ADR 0027); the cache removes the store's three.
- **On the same host, 0.43 gains 6-10%, and the cache 22-31%.** With the database next door, round trips are cheap and the gateway's Python sets the limit.
- **The 75 ms a governed key paid once a minute is gone:** the schema is now read again in the background, before it expires.
- **Every one of the 5.0 million requests returned 200** - or 201, for the writes.

Each cell is the median of 5 interleaved rounds, with its min-max range in the tables below; a difference counts only when the ranges don't overlap (v3's method). For MySQL, see Performance on MySQL, v2.

Contents

- Method
- Results
- Where the time goes
- Sizing, updated
- Limits
- Run it yourself

Method

v3's, unchanged: the same laptop (an Intel i7-12700H, on AC power, the `performance` profile and energy preference, a desk fan on it this time), the gateway on CPUs 0-7 (four performance cores), PostgreSQL 16 on 8-11, the load generator on 12-19; QueryAPIGate's own store a separate database on the same PostgreSQL server. One session, 2026-10-26 22:00 to 2026-10-27 00:28, right after the MySQL run.

LABEL	COMMIT
v0.42.0	`647ab6f`
0.43	`fe603c0` : `main` with everything 0.43 ships but its version number
0.43 + cache	the same, with `QUERYAPIGATE_METADATA_CACHE_TTL=5`

A `/health` floor before and after every block stayed between 1,978 and 2,323 requests a second. pgbench, straight to PostgreSQL: 39,579 lookups a second from one client, 130,738 from 32.

What v4 leaves out: scale-out across workers, the access log, mixed and hot-row writes. Nothing 0.43 changed affects them on their own, and v3's figures stand; the MySQL run measured scale-out on a PostgreSQL store and what logging costs with 0.43.

Results

One worker, 32 callers

CALL	V0.42.0	0.43	0.43 + CACHE
Lookup, plain key	786 (751-829)	852 (827-892)	1,012 (991-1,043)
Lookup, governed key	738 (713-771)	809 (803-856)	964 (956-992)
100-row page	682 (661-694)	725 (710-758)	835 (833-873)
Write	793 (764-836)	850 (832-887)	1,002 (999-1,046)

- **0.43 over 0.42: +6-10%.** The governed lookup and the page are clear; the plain lookup and the write overlap at their edges. What 0.43 saves here is CPU: the store-file check and the masking key's store read (#95), `db._target()`, and three of the four round trips a PostgreSQL call made (its statements now go as one message).
- **The governed key reads the store 3 times a call, not 4:** 0.42 also read the masking hash key every time.
- **The cache adds +15-19% on top of 0.43,** as in v3.

One caller at a time

CALL	V0.42.0	0.43	0.43 + CACHE
Lookup, plain key: p50	1.36 ms (1.34-1.41)	1.27 ms (1.23-1.30)	1.00 ms (0.99-1.01)
Lookup, governed key: p50	1.49 ms (1.44-1.51)	1.31 ms (1.27-1.35)	1.05 ms (1.03-1.06)

A governed call costs 0.05 ms more than a plain one on 0.43 + cache: the table check and masking, both reused from cache.

Latency against offered load

Open loop, latency corrected, the plain lookup at a share of each build's capacity (0.43: 892 a second; with the cache, 1,042):

OFFERED	0.43: P50 / P99 / P99.9	0.43 + CACHE
25%	2.1 / 3.0 / 11.0 ms	1.4 / 2.1 / 10.4 ms
50%	1.5 / 2.8 / 7.5 ms	1.1 / 2.1 / 9.3 ms
75%	1.4 / 4.6 / 14.2 ms	1.1 / 4.3 / 13.0 ms
90%	1.6 / 15.9 / 20.5 ms	1.1 / 13.3 / 19.0 ms
100%	64 / 108 / 110 ms	88 / 141 / 150 ms

As in v3: **p50 is flat to capacity, and p99 holds at about 4-5 ms to 75%,** then climbs. Plan at 75%.

The cold path

One 150 s run of the lookup at 8 callers, each request's duration from gunicorn's access log, 0.43 - the test that found the once-a-minute spike in v3:

KEY	REQUESTS	P50	P99	P99.9	P99.99	SLOWEST
governed	131,380	7.59 ms	10.82 ms	12.96 ms	**22.33 ms** (v3: 73.7)	138.8 ms, the first call
plain	131,380	7.62 ms	10.92 ms	13.60 ms	18.89 ms	22.0 ms

No second after the first stands out. In v3 the governed key paid 76 and 74 ms at t=61 s and t=121 s, when its copy of the schema expired and the next call read it again and traced its statement again. 0.43 reads it again in the background after 45 s, and keeps the traces when the schema hasn't changed. The very first call still reads the schema itself: 139 ms, once, per connection and process.

A database across a network

netem delays everything the PostgreSQL server sends by 0.5 ms - the data database and QueryAPIGate's store, which share it here, as a deployment with both in another zone would:

	V0.42.0	0.43	0.43 + CACHE
One caller: p50	6.58 ms (6.47-6.64)	4.39 ms (4.32-4.51)	**1.60 ms** (1.57-1.78)
One caller: calls/s	153 (150-156)	226 (220-233)	604 (558-615)
32 callers: calls/s	826 (770-832)	888 (829-892)	1,029 (999-1,033)

- **For one caller, every round trip shows:** 0.43 saves three of the seven (2.2 ms), and the cache the other three that weren't the query (another 2.8 ms).

- **With 32 callers, 0.43 gains less (+7%, at the edge of the range):** eight threads already hid most of the waiting, and the worker's CPU was the limit. The cache, which also saves CPU, still adds 16%.

Threads per worker

0.43, the plain lookup, 32 callers:

THREADS	1	2	4	8 (THE DEFAULT)
Calls/s	904 (888-922)	**1,012** (960-1,016)	928 (880-939)	896 (839-903)

Two threads are still best with the database on the same host (13% faster than eight), as in v3. Set them with `QUERYAPIGATE_THREADS` (from 0.43).

Where the time goes

Round trips per call

One caller's p50 against delay, calibrated by psycopg2 sending a known one statement (1.25 ms of p50 per ms of delay):

	P50 AT +0 / 0.5 / 1 / 2 MS	SLOPE	ROUND TRIPS
psycopg2, 1 statement	0.02 / 0.67 / 1.23 / 2.53 ms	1.25	1
psycopg2, 4 statements	0.05 / 2.51 / 4.80 / 9.90 ms	4.87	3.9
v0.42.0	1.38 / 6.62 / 10.22 / 19.44 ms	8.90	**7.1**: 4 to the data database, 3 to the store
0.43	1.26 / 4.54 / 6.86 / 11.92 ms	5.25	**4.2**: 1 and 3
0.43 + cache	0.98 / 1.67 / 2.36 / 3.46 ms	1.24	**1.0**: the query alone

With the cache, a call on 0.43 makes one round trip: the query, sent as one message that also sets its access mode and time limit, and resets whatever a previous query left on the connection.

The CPU

py-spy on one worker, the plain lookup at 32 callers:

	0.43	0.43 + CACHE
Store layer: the key, the query and the connection	22.9%	4.5%
Logging	21.2%	**25.2%**
gunicorn, sockets, HTTP	18.7%	22.3%
Flask and Werkzeug	10.8%	14.1%
Data query: psycopg2 and the pool	7.6%	10.5%
QueryAPIGate's request handling	5.6%	6.9%
Everything else	13.2%	16.5%

With the cache on, logging is the largest single cost of a call - a quarter of the samples, much of it threads waiting for the logging module's lock. Turning off gunicorn's access log and QueryAPIGate's request lines (`QUERYAPIGATE_ACCESS_LOG=off`, `QUERYAPIGATE_LOG_REQUESTS=off`) was worth 21% on MySQL.

Sizing, updated

v3's rule, with 0.43's figures:

- **CPU:** about 670 fast calls a second per worker, or 780 with the metadata cache, at 75% of this run's capacity, where p99 is 4-5 ms. (v3: 550 and 650.)
- **Waiting:** threads ÷ (query time + round trips × network round-trip time), with **4 round trips** a call (1 to the data database, 3 to the store) on 0.43, and **1** with the cache. (v3: 7 and 4.)
- **Workers needed** = the larger of the two limits divided into your calls a second, plus one spare.

At 0.5 ms away, one worker with 8 threads can wait on 8 ÷ (0.03 + 4 × 0.5 ms) = about 3,900 calls a second on 0.43, and 15,000 with the cache - neither near the CPU figure. **Across a network, 0.43 is CPU-bound where 0.42 was close to being bound by waiting.** Tune a deployment for throughput puts these together with MySQL's.

Limits

As v3's: one machine, a laptop's power limit, a simulated network (one-way delay, no jitter or loss), cheap queries on data that fits in memory, and no soak test. Scale-out, the access log and mixed writes weren't rerun.

Run it yourself

```shell
pip install -e ".[server,postgres]" py-spy
python benchmarks/oltp.py --out /tmp/qag-oltp --base v0.42.0 setup
python benchmarks/oltp.py --out /tmp/qag-oltp --base v0.42.0 env pgbench b1 b2 b3 b5 b7 b9 b10 b11 env
# 2.5 hours
python benchmarks/oltp_report.py /tmp/qag-oltp/results.jsonl
```

The 2026-10-26 run is in `benchmarks/results/oltp-postgres-2026-10-26/`.